LDR (LIGHT DEPENDENT RESISTOR) EXERCISE

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Simulink Set-Up and LDR Testing.....2
4.0 MATLAB Function and LDR Testing.....3
5.0 Exercises.....4
6.0 Concluding Remarks.....4
Recommended Citation.....4

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Map the analogue input reading from a light dependent resistor (LDR) to a suitable range and use it to control LED brightness via pulse width modulation (PWM).	☐
2	Use a MATLAB Function block to implement logic that switches the LED on or off based on the measured LDR values.	☐

2.0 Hardware Set-Up

The exercise involves connecting an LED to a PWM Arduino output pin and controlling its brightness using the readings from an LDR, as shown in Figure 1.

Note: While the ACE-Box can be used for this exercise, it is not required. Only the individual components listed below are needed.

Required hardware

- ATmega328P microcontroller (Arduino Uno)
- USB Type-B cable
- Breadboard
- LED
- 2 × 220 Ω resistors
- LDR (light dependent resistor)
- 5 × male-to-male breadboard wires

Hardware Assembly Steps

1. Place the LED on the breadboard and connect a 220 Ω resistor to its anode.
2. Connect the LED’s cathode to GND, and the free end of the resistor to Pin 9.

3. Place the LDR on the breadboard and connect one of its legs to 5 V.
4. Connect a 220 Ω resistor from the second LDR leg to GND to form a voltage divider.
5. Connect the junction between the LDR and the resistor to A0 on the Arduino.

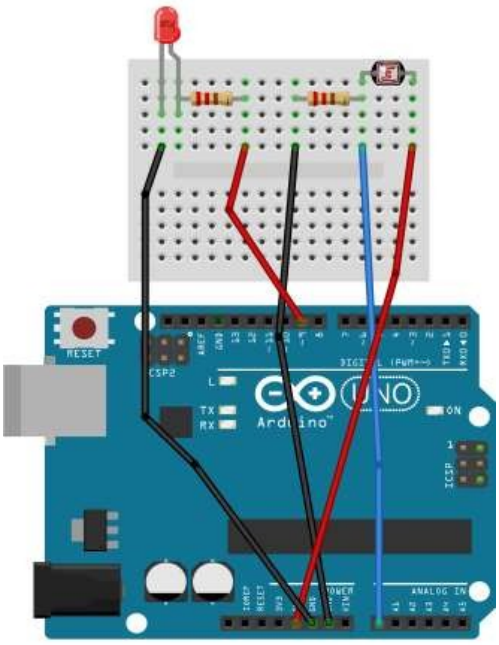


Figure 1: *Hardware Set-Up for the LDR (Light Dependent Resistor) Exercise*

3.0 Simulink Set-Up and LDR Testing

In this part of the exercise, you will develop a Simulink model that controls the brightness of an LED using PWM, based on the analogue reading from the LDR: as the analogue reading increases, so does the brightness of the LED, as shown in Figure 2.

Simulink Model Construction Steps

1. Add an Analog Input block, set the Pin number to A0 and the Sample time to 0.01 s. This reads the sensor voltage and outputs a value between 0 and 1023.
2. Insert a Gain block with a value of 255/1023. This scales the ADC reading into the 0–255 range needed for the PWM duty cycle.
3. Connect the scaled signal to a PWM block and set the Pin number to 9. The PWM block uses the 0–255 value to generate a proportional PWM signal on the Arduino.
4. Connect two Display blocks, one to the Analog Input output and one after the Gain block. These show the raw sensor reading and the scaled PWM value in real time, so that you can verify the conversion.
5. Deploy the model to the Arduino. Vary the light level falling on the LDR and observe the PWM value and the LED brightness change.
6. Note that the LDR-to-LED mapping may operate in the opposite direction to what is wanted: a lower LDR value (darker conditions) produces a lower LED brightness, whereas the LED would normally be expected to brighten as the environment darkens. The next part of the exercise addresses this.

To recap how to run Simulink code generation and deploy the model to the Arduino Uno, see Section 4.0 of the LED (Digital) Exercise or visit Code Generation.

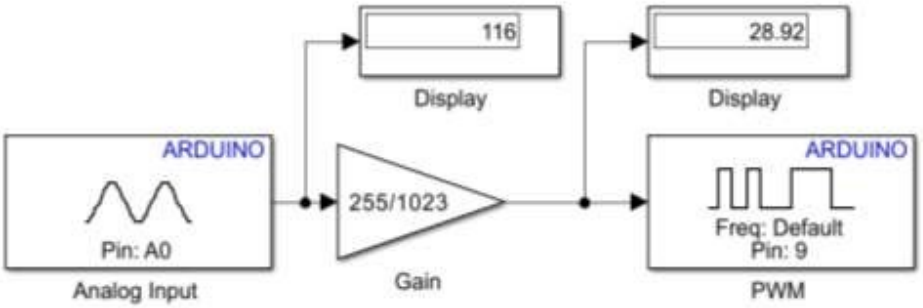


Figure 2: *Simulink Set-Up for the LDR (Light Dependent Resistor) Exercise*

If you encounter any errors, see Error Messages for troubleshooting help.

4.0 MATLAB Function and LDR Testing

The model is now updated to include a MATLAB Function block after the Gain block. Instead of sending the scaled value straight to the PWM output, the signal is passed through a short piece of code that decides whether the output should be on or off, as shown in Figure 3.

Simulink Model Construction Steps

1. Add a MATLAB Function block between the Gain and PWM blocks. This applies simple decision-making to the signal before it reaches the PWM output.
2. Open the MATLAB Function block and replace the default code with the code below.
3. Deploy the model. The LED now follows the scaled signal only while it is at or below half of its range; above this threshold the output is switched off, creating simple on/off threshold behaviour.

MATLAB Function: threshold logic for the LED (fcn)

```
function y = fcn(u)

if u <= cast(255/2,'like',u)
    y = u;
else
    y = cast(0,'like',u);
end
```

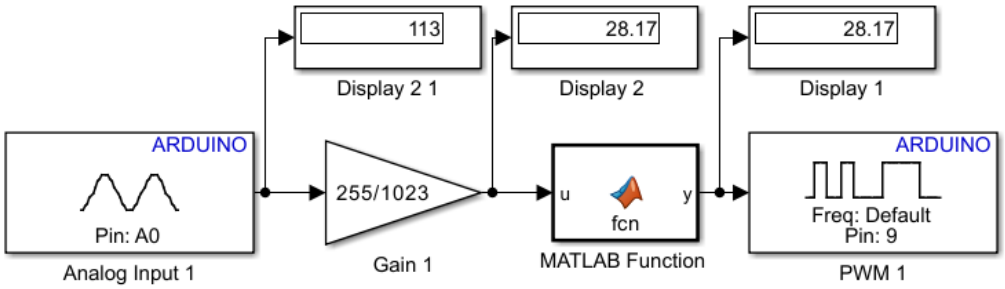


Figure 3: *MATLAB Function Set-Up for the LDR (Light Dependent Resistor) Exercise*

Understanding the MATLAB Function Block

- u is the input from the Gain block (0–255).
- 255/2 is the midpoint (127.5).
- If the input is less than or equal to the midpoint, the value passes through to the PWM output.
- If the input is above the midpoint, the output is forced to zero.

5.0 Exercises

1. Modify the Gain or MATLAB Function logic so that the LED becomes brighter as ambient light decreases.
2. Replace the fixed midpoint (255/2) with a tuneable threshold parameter and evaluate its effect.
3. Record ADC readings under different lighting conditions (e.g., room light, shaded, phone torch) and comment on the sensitivity of the sensor.

6.0 Concluding Remarks

This exercise has demonstrated analogue sensing using a voltage divider, signal scaling for PWM actuation, and basic decision-making using a MATLAB Function block. It reinforces the complete signal chain: sensor → scaling → logic → actuator.

You have explored both proportional control (continuous brightness adjustment) and threshold-based control (on/off behaviour), two common embedded control strategies.

Before completing the exercise, return to the Learning Outcomes in Table 1 and confirm that you are confident you can demonstrate each one. If so, tick the corresponding Achieved box.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). LDR (Light Dependent Resistor) Exercise. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.23885012